Vehicle Embedded Control System Timeline

1. Executive Summary & Objective

This document outlines the confirmed, phased engineering roadmap for the Vehicle Embedded Control System (VECS). The primary objective of this timeline is to establish complete digital control over the motorcycle's chassis electronics, implement a robust local Controller Area Network (CAN) backbone, and deploy a secure cloud telematics pipeline. All milestones listed below must be fully integrated, tested, and completed by **September 10, 2026**.

2. Phase-by-Phase Development Timeline

Phase I: Core Chassis Automation (BCM)

- **Objective:** Achieve absolute control over the vehicle's low-voltage electrical systems.
- **Deliverables:** Design, prototype, and program the **Body Control Module (BCM)**.
- **Scope:** The BCM will assume complete handling of all body electronics, including switches, lighting systems, power distribution, and safety interlocks.
- **Validation:** Thorough bench testing and physical integration to guarantee 100% reliability of low-voltage switching before introducing data networking.

Phase II: Mobile Infrastructure & Protocol Handshake (VECS Android App v1.0)

- **Objective:** Build the foundational mobile application layer designed to interface with the vehicle's future cloud gateway.
- **Deliverables:** Deploy a lightweight, stable **VECS Android Application**.
- **Feature Set (Initial MVP):**
  - State of Charge (SOC) telemetry display.
  - Estimated vehicle range calculation.
  - "Ping My Bike" location tracking beacon.
- **Validation:** The primary metric for this phase is validating reliable, low-latency communication endpoints. Features will remain intentionally simple to ensure connection stability before future feature scaling.

Phase III: Cloud Gateway Deployment (TCU)

- **Objective:** Establish a secure, high-uptime internet gateway for the vehicle.

- **Deliverables:** Design and develop the **Telematics Control Unit (TCU)** hardware and firmware.
- **Network Topology Architecture:** The TCU is designated as the **sole** firewall and internet-facing ECU on the vehicle. Direct external cloud connections from any other module are strictly prohibited.
- **Validation:** End-to-end testing between the TCU firmware, cloud database/broker, and the VECS Android App to verify a bulletproof handshake and data packet delivery.

Phase IV: Network Physical Layer Design (CAN Bus Backbone)

- **Objective:** Construct the physical communication infrastructure linking all current and future ECUs.
- **Deliverables:** Design and route a dedicated linear **CAN Bus network**.
- **Specifications:** The bus topology must run continuously from the physical rear end to the front of the motorcycle, incorporating twisted-pair wiring and proper termination resistance ($120\ \Omega$) at its boundary nodes.

Phase V: Infrastructure Overhaul (Accessory Wiring Harness)

- **Objective:** Eliminate legacy wiring vulnerabilities and integrate the new data backbone.
- **Deliverables:** Completely strip and replace the bike's electrical accessories wiring harness.
- **Scope:** Clean routing of low-voltage power lines alongside the newly minted Phase IV CAN Bus lines, creating modular, plug-and-play drops for all ECUs.

<u>3. Milestone Target State: Deadline September 10</u>

By **September 10**, the motorcycle's electrical ecosystem must achieve the following integrated operational architecture:

[VECS Android App]

▲

│ (Cellular/MQTT/Cloud Internet)

▼

[Telematics Control Unit (TCU)]

▲

│ (New Dedicated CAN Bus Backbone)

▼

[Body Control Module (BCM)] ——► [Complete Body Electronics & Accessories]

1. **Chassis Control:** The BCM maintains flawless, deterministic control over all motorcycle body electronics and accessory circuits.
2. **Telematics Pipeline:** The TCU successfully bridges vehicle data to the internet, maintaining a strong, stable telemetry stream with the VECS Android App.
3. **Local Networking:** The BCM and TCU actively communicate with each other over the newly deployed physical CAN Bus infrastructure, utilizing standardized message frames to pass local switch and system states.